



git



- Lernziele
- Git – Kurzgeschichte
- Problemstellung
- Fiktives Beispiel
- Lösung
- Funktionsweise / Arbeitsablauf
- Aufgaben

Lernziele

- Wissen was Git und Github ist
- Ein Repository erstellen können
- Pushen, fetch, pull, commit kennen und anwenden können
- Mit Github Desktop arbeiten können
- Branches erstellen und wechseln können

Git

- Versionsverwaltungstool
- Entwicklungsstart von Linus Torvalds am 3. April 2005
- 1.0 Release: 21 Dezember 2005
- *"I'm an egotistical bastard, and I name all my projects after myself. First '[Linux](#)', now 'git'."*
 - Git = unpleasant person (British English slang)
 - Git = Idiot

Problemstellung

- Meistens entwickelt ein Entwicklerteam Software
 - Was ist die aktuelle Softwareversion?
 - Wie überschreibt man kein Code unabsichtlich?
 - Wie handhabt man unterschiedliche Versionen?
- Features
 - Wie werden neue Features eingespielt?
 - Was passiert mit Code als Prototyp für ein Feature? (POC)
- Rückverfolgbarkeit
 - **Wer** hat **welche** Änderungen mit **welcher** Intention vorgenommen?

Fiktives Beispiel

- Schulsoftware für Schüler und Schulangestellte
- Verwaltung von Noten, Schulplänen, Notizen, Schülerabgaben, Löhne etc.
- Software wird gebraucht von: Lehrpersonen, Hausdienst, Sekretariat, Bereichsleiter, Schuldirektion, Schüler etc. => Jeder Person bei der Schule
- Ausgangslage: Unsere Software ist im Einsatz bei drei Schulen:
 - BFO Visp
 - OS Visp
 - Kollegium Spiritus Sanctus Brig
- Wir als Klasse arbeiten alle gemeinsam an dieser Software

Fiktives Beispiel

- Wir als Klasse arbeiten alle gemeinsam an dieser Software
- **Herausforderung:**
- Wie handhaben wir, das wir alle gemeinsam daran arbeiten können?
- Wie teilen wir die Daten?
- Muss jeder immer zur selben Zeit online sein?
- Wo sind die Daten?
- Wer hat immer die aktuellste Version?
- Was wenn ein Schüler (z.B. wegen Krankheit) ausfällt und wir genau diese Arbeit von dem Schüler brauchen?

Fiktives Beispiel

- Wir bieten 5 Jahre Support für ältere Versionen unserer Anwendung an.
- BFO Visp hat die aktuellste Version(v3), OS Visp ist auf der ältesten Version (v1) und hat noch 1 Jahr Support und Kollegium Spiritus Sanctus Brig ist auf v2 und hat noch 2 Jahre Support.
- **Herausforderung:**
- Wie handhaben wir die Unterschiedlichen Versionen?
- Was geschieht, wenn wir in v1 einen kritischen Fehler (=Bug) finden, welcher in jeder anderen Version auch gefixt werden muss?
- Was machen wir, wenn sich OS Visp (v1) auf einmal entscheidet, für die aktuellste Version zu zahlen?
- Wo speichern wir die unterschiedlichen Versionen?

Fiktives Beispiel

- Dank guter Mundpropaganda haben wir 10 neue Kunden(=Schulen).
- Jede Schule hat aber ein paar Sonderwünsche:
- Wunsch 1: Eine Schule möchte v2 offline installieren, später eventuell auf v3.
- Wunsch 2: Eine Schule möchte eine Testphase haben und kann sich nicht entscheiden, ob der Preisaufschlag für v3 zu v2 gerechtfertigt ist. Darum möchten sie gerne beide Versionen testen.
- Wunsch 3: Drei Schulen wünschen sich ein spezielles Feature. Die Schulen möchten gerne jeweils einen POC (Proof of Concept). Eine Schule möchte den POC aufbauend auf v1, die andere auf v2 und die letzte Schule ausgehend von der jeweils aktuellsten Version.
- **Herausforderung:**
- Wie gehen wir mit unterschiedlichen POCs um?
- Wie können wir Arbeit so aufteilen, das n-Schüler an einem POC arbeiten können?

Fiktives Beispiel

- **Andere Herausforderungen:**
- Wie können wir v4 vorbereiten?
- Wie managen wir Features, welche wir für einen Kunden vorbereitet haben, jedoch nicht mehr gebraucht werden?
- Wie handhaben wir alle Herausforderungen wenn wir noch n-andere Software bereitstellen?
 - Party app
 - Matura lernhilfe app
 - ...

Lösung ?



New.psd



NewFinal.psd



NewFinal
Final.psd



NewFinal
Latest.psd



NewFinal
ForSure.psd



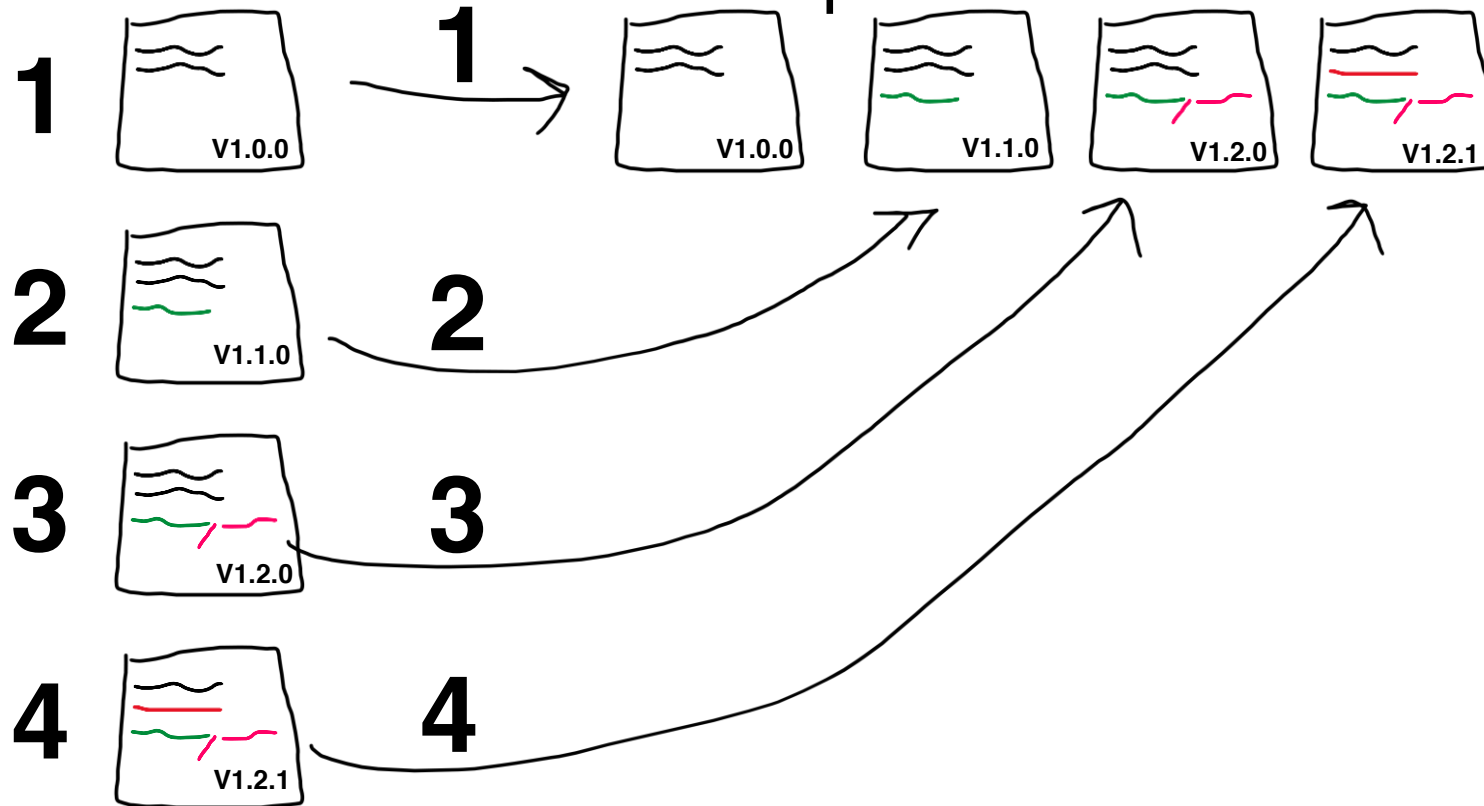
NewFinalFor
SureLatest.psd

Lösung



«Alter Weg»

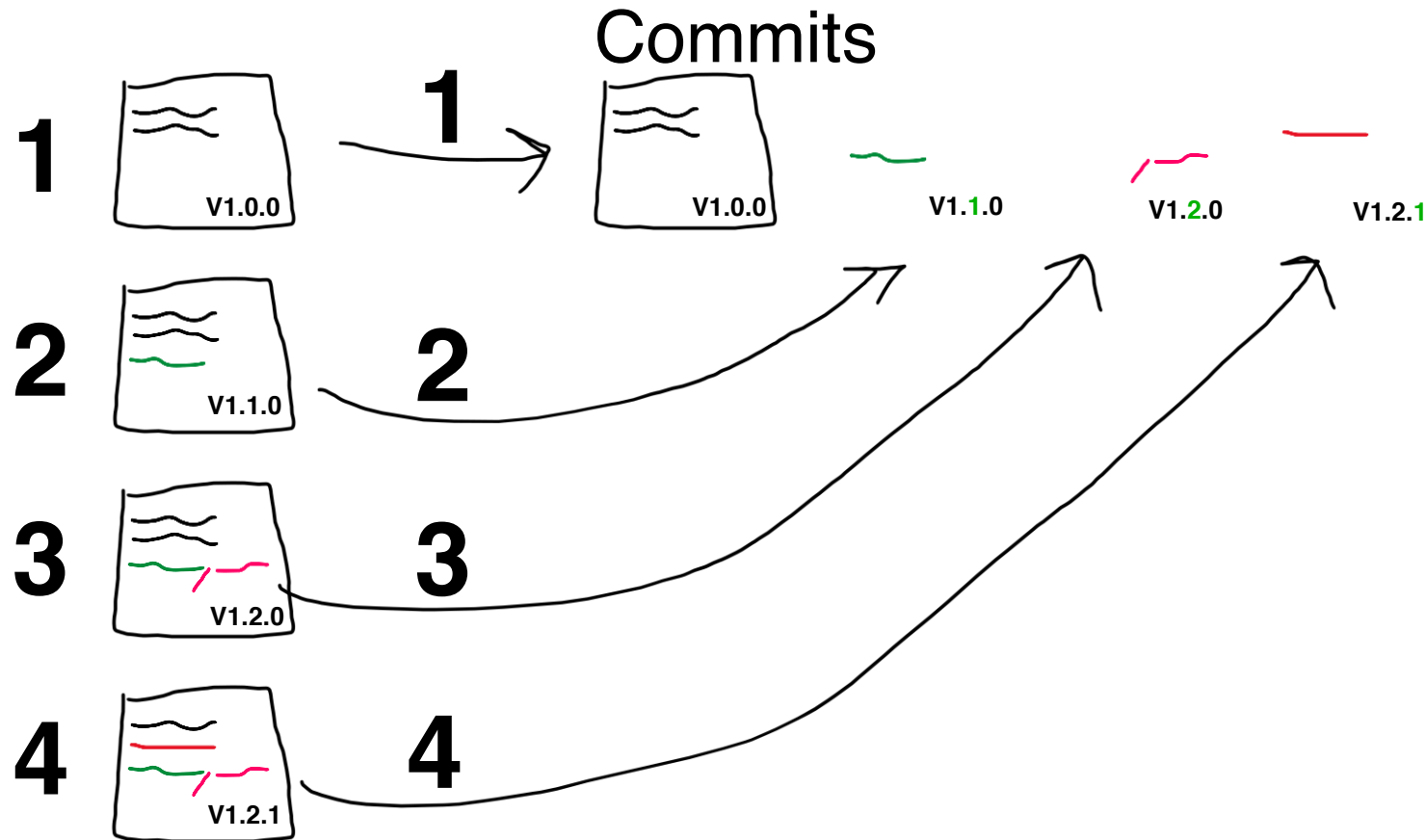
Backups



Herausforderungen:

- Der Backup-Ordner wird viel Speicherplatz in Anspruch nehmen, da dort immer wieder neue Dateien angelegt werden, welche aber sehr ähnlich zu den anderen Dateien ist
- Vorgenommene Änderungen zu sehen ist schwierig
- Die Gefahr, vorherige Dateien zu ändern, besteht
- Falls man ausgehend von einer Datei n-verschiedene Ansätze versuchen will und die Ansätze behalten will, muss man auch diese speichern.
- Änderungen in einer älteren Version müssten systematisch durch alle neueren Versionen durchgezogen werden
- Und viele andere Herausforderungen!

Git

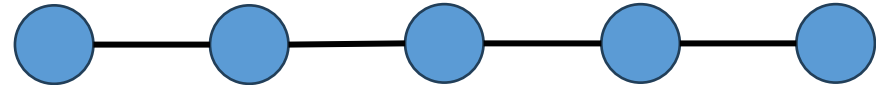


Mit Git speichern wir nur die tatsächlichen Änderungen (=Commits).

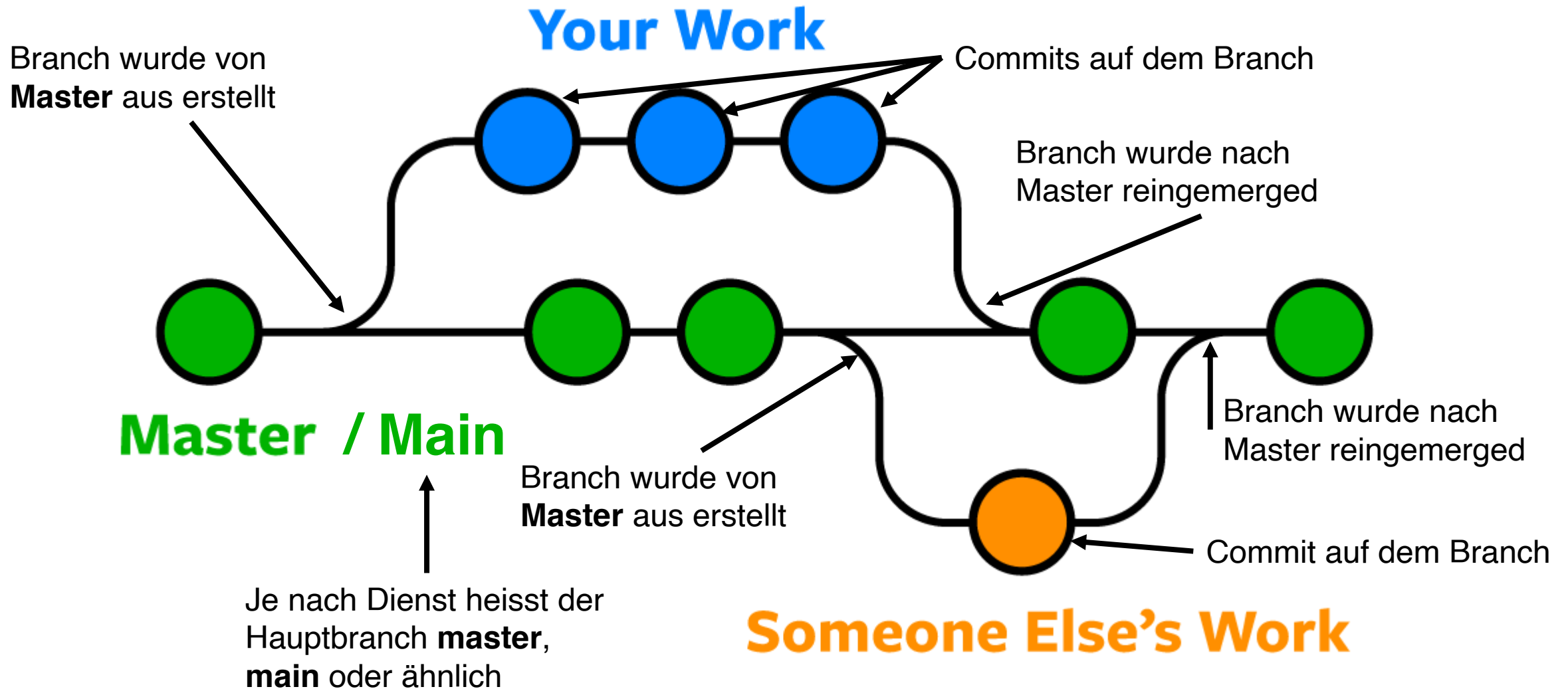
Dies braucht nicht nur weniger Speicherplatz, wir können auch bequem jeglichen Stand anhand der Commits wiederherstellen!

Git commits

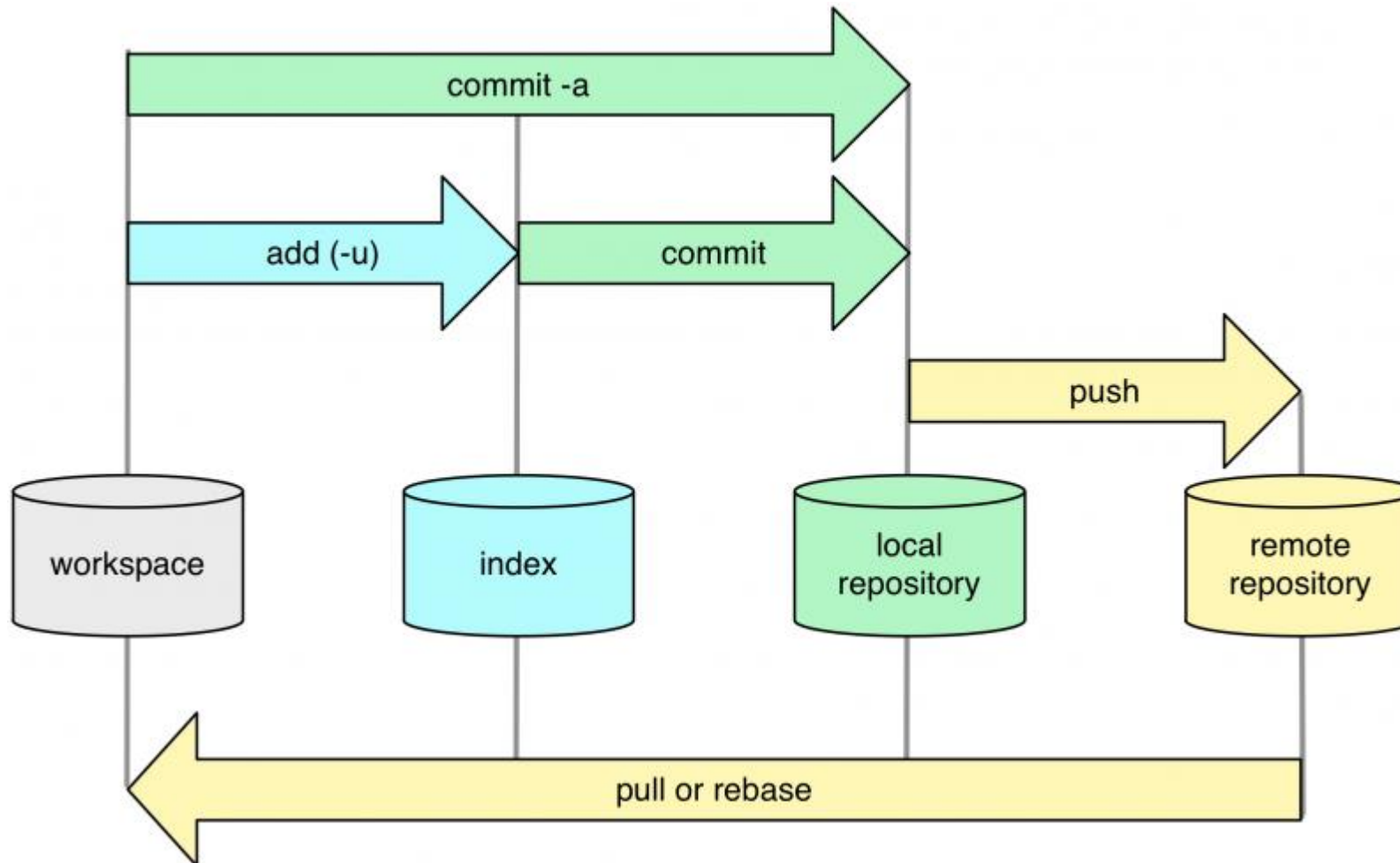
- Ein Commit dient als Schnappschuss vom Projekt zu einem bestimmten Zeitpunkt.
- Ein Commit speichert den aktuellen Zustand aller verfolgten Dateien im Repository (=Behälter)
- Jeder Commit enthält eine eindeutige ID (Beispiel: 521747298a3790fde1710f3aa2d03b55020575aa)
- Ein Commit enthält unterschiedliche Informationen:
 - Vorgenommene Änderungen
 - Author
 - Zeitpunkt
 - Commitmessage (Nachricht, was gemacht wurde)
- Commits werden meistens als Punkte dargestellt



Funktionsweise Git Branches



Funktionsweise / Arbeitsablauf



Github

- Baut auf git auf
- Repository Hosting Service
- Public, private repos (= Abkürzung von Repositories)
- Gibt auch andere Dienste:
 - Gitlab
 - Bitbucket
 - Git Kraken
 - Etc...

Demo Github (Desktop)

Begriffe

- Repository = Behälter (Das Projekt)
- Commit = «Stempel», Was habe ich gemacht?
- Add / Staging = File(s) hinzufügen
- Push = Zum remote Repository hochladen
- Fetch = Lokaler Branch mit remote Branch abgleichen
- Pull = Fetch und Änderungen holen / mergen
- Merge = Änderungen zusammenführen
- Merge conflict = Widersprüchliche Änderungen in verschiedenen Branches, die nicht automatisch zusammengeführt werden können. Manuelle Auflösung erforderlich
- Branch = Ein «Zweig»; Wird für neue Features / Bugfixes gebraucht
- Auschecken = (check out) Branch wechseln

Git

- Download git
 - <https://git-scm.com/download/win>
- Download Github Desktop
 - <https://desktop.github.com/>
 - Grafische Oberfläche
- Bei Github ein Account erstellen
 - <https://github.com/>
 - Empfehlung: Private E-Mail
- Github Desktop: Einloggen mit Github Account

Links

- Dokumentation git
 - <https://git-scm.com/docs>
- Tutorial auf Deutsch
 - <https://rogerdudler.github.io/git-guide/index.de.html>
- Introduction to GitHub
 - <https://github.com/skills/introduction-to-github>

Github Limits (Stand 10.2024)

- Pro Commit maximum ~100 MB
 - Eine Datei, z.B. ein .mp4, kann von 90 MB in einem Commit auf über 100 MB in einem anderen Commit ohne Probleme wachsen
- Pro Monat maximum ~1 GB
 - Falls das Limit erreicht wurde, könnt ihr nicht mehr pushen!
 - Daher aufpassen bei grossen Dateien wie Videos, Assets (3D Programme) etc.
- See:
 - <https://docs.github.com/de/get-started/learning-about-github/githubs-plans>
 - <https://github.com/pricing>

Fragen?

Aufgabe 1

1. Erstellt ein lokales Repository
2. Erstellt einen Commit mit «Initial commit» und pusht das repo auf Github
 1. Github Desktop erstellt den Commit für euch selbstständig
3. Erstellt eine Textdatei und schreibt «Meine ersten Schritte mit git» in die Textdatei
4. Fügt die Textdatei hinzu mit dem commit «Add first text file»
5. Fügt eine weitere Textdatei hinzu und committed auch diese
6. Pusht alles zusammen auf Github
7. Kontrolliert auf Github, ob eure Änderungen da sind
 1. Schau dir deine einzelnen Commits an
8. Löscht lokal euer Ordner mit den zwei Textdateien
9. Ladet entweder per Github oder Github Desktop euer repo wieder herunter
10. Fügt nun ein Bild hinzu und committed auch dies Sinnvoll
11. Überprüft erneut auf Github, ob euer commit angezeigt wird

Aufgabe 1

12. Erstellt einen neuen Branch «TestFeature» basierend auf eurem main-Branch und checkt ihn aus
13. Löscht hier die erste Textdatei, committed und pusht diese Änderung
14. Wechselt nun zurück auf den main-Branch und verifiziert, dass auf dem main-Branch die Textdatei immer noch vorhanden ist
 1. Fasse in eigenen Worten zusammen, warum die Textdatei noch vorhanden ist
15. Erstellt nun erneut einen neuen Branch «TestMerge» basierend auf eurem main-Branch und checkt ihn aus
16. Verifiziert, dass auf dem Branch «TestMerge» die Textdatei noch vorhanden ist
17. Merged nun den Branch «TestFeature» in den Branch «TestMerge»
18. Publiziert eure Branches auf Github und schaut euch den aktuellen Stand jedes Branches an sowie die Commits von jedem Branch

Aufgabe 2

1. Bevor ihr diese Aufgabe macht: Macht zuerst ein Backup mit euren Lösungen / Dateien / Notizen zu diesem Kurs!
 1. => Kopiert euren Ordner
2. Erstellt ein neues Repository: «Emvs1_Java_Grundlagen» (oder einen ähnlich passenden Namen)
3. Fügt eure Kursdokumente in das Repository hinzu und committed sowie pusht eure Änderungen
4. Macht eine Codeänderung und überprüft, das es auch in git eine Änderung anzeigt (Diese müsst ihr nicht committen/pushen)
5. Zeigt mir euer Repository

Aufgabe 3

1. Beispiel auf nächster Folie
2. Bildet zweier, maximal dreier Gruppen. Jede Person erstellt ein neues Testrepository und lädt die anderen Gruppenmitglieder in das Repository als Collaborator ein.
3. Der Inhaber des Repository erstellt für jedes Gruppenmitglied einen neuen Branch. Der Branchname ist dabei der Name des jeweiligen Gruppenmitglieds
4. Die Gruppenmitglieder checken den für sich vorhergesehenen Branch aus und fügen eine Textdatei mit ihrem Namen als Inhalt hinzu. Committed diese und Pusht diese.

Aufgabe 3, Beispiel

- Gruppe Hanna und Hans
- Hanna besitzt ein Repo mit Hans als Collaborator
- Das Repo von Hanna hat zwei Branches: der main-Branch sowie der Branch «Hans»
- Hans hat im Branch «Hans» eine Textdatei mit dem Inhalt «Hans» erstellt sowie gepusht
- Hanna hat in ihrem Branch (main) eine Textdatei mit dem Inhalt «Hanna» erstellt sowie gepusht
- Die Schritte oben nun nochmals umgekehrt für Hans

Aufgabe 4

1. Erarbeite selbstständig, was eine .gitignore –Datei und was ihr Zweck ist.
2. Erstelle eine neue Datei «SuperSecret.txt» und füge sie dem .gitignore hinzu
3. Erstelle einen neuen Ordner und füge den gesamten Ordnerinhalt dem .gitignore hinzu (und zwar so, dass du dies nur einmal angeben musst, unabhängig davon wie viele Dateien im Ordner sind)
4. Überprüfe, dass deine Datei nie online zum Repository gepusht wird
5. Zeige mir deinen Aufbau von dem .gitignore

Aufgabe 5

- Links durchgehen

Ab sofort

- Eure Arbeitsschritte committen und pushen
- Euer Code sollte nun immer online sein (Aktuell)
- Lieber häufig als gar nicht committen und pushen
- «**USB-Stick / Festplatte verloren**» gibt es nicht mehr
 - Eure Änderungen sollten nun **immer** online sein

In case of fire



1. git commit



2. git push



3. leave building